



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

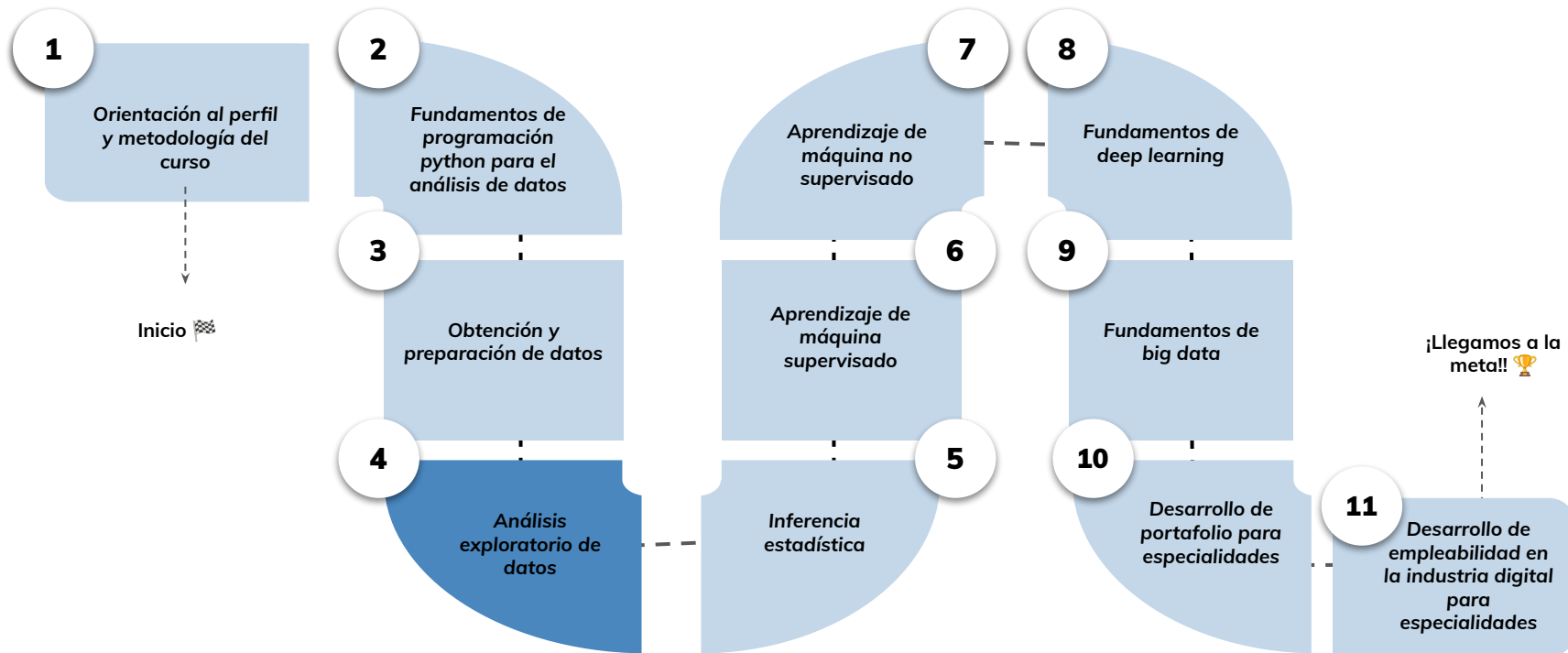


› Librería matplotlib- Parte I

Aprendizaje Esperado 6: Aplicar personalizaciones a los gráficos de datos utilizando la librería matplotlib para la resolución de un problema dado.

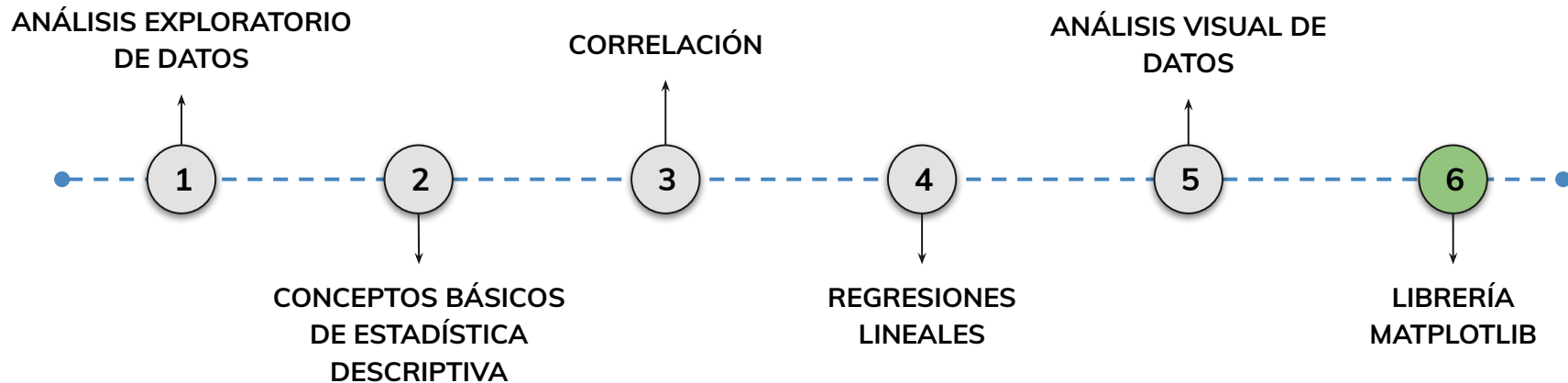
Hoja de ruta

¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

6.

Visualización de datos con Matplotlib

En esta clase conocerás los fundamentos de la biblioteca Matplotlib, aprendiendo a representar variables numéricas y categóricas mediante gráficos básicos. Verás cómo ajustar etiquetas, estilos y títulos para que tus gráficos sean claros y útiles.

Introducción a
matplotlib.pyplot

Gráficos de líneas,
barras

histogramas y
boxplots

colores, ejes y
títulos

Subplots para
múltiples gráficos



Objetivos de aprendizaje

¿Qué aprenderás?



- Comprender el funcionamiento básico de la librería Matplotlib.
- Crear gráficos simples: líneas, barras, histogramas y boxplots.
- Personalizar gráficos con etiquetas, colores y estilos.
- Utilizar subplots para mostrar más de un gráfico en una sola figura.
- Exportar gráficos en distintos formatos.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Aprendimos a realizar visualizaciones multivariadas con Seaborn.
- Implementamos FacetGrid y heatmap para análisis exploratorio avanzado.
- Aplicamos estilo visual para mejorar la interpretación de gráficos.
- Trabajamos con datasets categóricos y numéricos a la vez.
- Visualizamos correlaciones entre múltiples variables en una sola figura.

Librería matplotlib



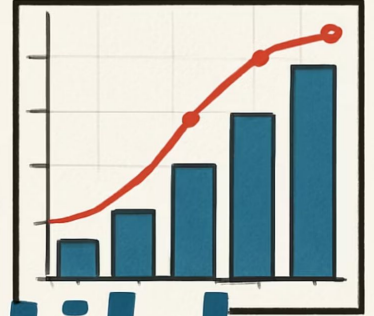
Visualización de Datos con Matplotlib

La visualización de datos es una de las habilidades más esenciales en el análisis de datos, ya que permite representar gráficamente información compleja de forma clara, comprensible y atractiva. En este contexto, la librería **Matplotlib** se posiciona como una de las herramientas más robustas y versátiles dentro del ecosistema Python. Permite generar gráficos de calidad profesional, con amplias posibilidades de personalización, desde simples líneas hasta representaciones más complejas como diagramas de dispersión o boxplots.

Características de la librería Matplotlib

Matplotlib es una librería de visualización bidimensional escrita en Python. Fue desarrollada con la intención de replicar la funcionalidad de MATLAB en cuanto a gráficos, pero con la ventaja de ser completamente open source. Lo que la hace especialmente poderosa es su capacidad de generar gráficos con un alto grado de personalización, además de permitir su integración con otras bibliotecas como NumPy, Pandas y Seaborn.

*mat-
plotlibl*



Versatilidad de Matplotlib

Enfoque dual

A diferencia de otras herramientas, Matplotlib puede utilizarse tanto desde un enfoque orientado a scripts simples como desde una estructura más robusta basada en objetos. Esto la convierte en una opción apta tanto para principiantes como para desarrolladores avanzados que requieren un control más fino sobre cada componente del gráfico.

Exportación flexible

Una de sus principales ventajas es que permite exportar los gráficos en múltiples formatos de alta resolución (como PNG, PDF, SVG, entre otros), siendo útil no solo para notebooks interactivos sino también para presentaciones y publicaciones.

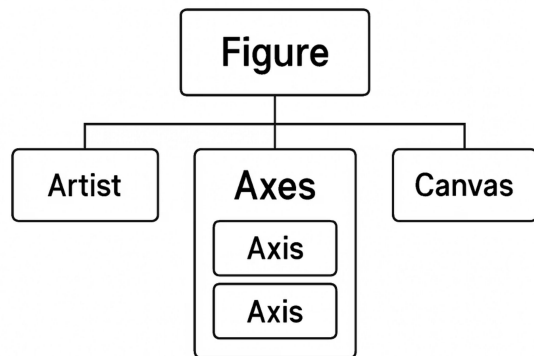
Importación de la librería

```
import matplotlib.pyplot as plt  
import numpy as np
```

Para comenzar a trabajar con Matplotlib, es necesario importar el módulo pyplot, que agrupa la mayoría de las funciones necesarias para crear gráficos. La convención más habitual es hacerlo como plt para simplificar el código.

Este módulo funciona como una interfaz de estilo procedural, similar a MATLAB. Si bien existe un enfoque más estructurado basado en clases, pyplot ofrece una manera más rápida y sencilla de comenzar a graficar. En la mayoría de los casos se complementa con NumPy para generar o manipular datos numéricos.

Matplotlib Figure Hierarchy



Componentes principales de un gráfico

Cada gráfico de Matplotlib está compuesto por una estructura jerárquica de objetos. En la base de esta jerarquía se encuentra la **Figure**, que actúa como el lienzo general sobre el cual se dibujan los distintos elementos.

Dentro de la figura se encuentran los **Axes**, que son áreas individuales de graficado, donde se colocan las líneas, barras, puntos u otros elementos.

Estructura de componentes

Cada Axes contiene sus propios ejes (X e Y), etiquetas, ticks, leyendas, títulos y otros componentes gráficos. Esta separación permite una gran flexibilidad, ya que una misma figura puede contener múltiples áreas de graficado independientes.

El modelo de Matplotlib se puede representar de la siguiente manera:

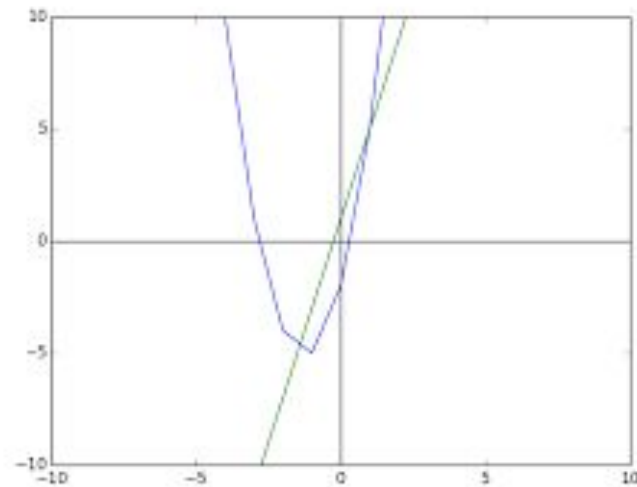
Componentes y sus funciones

Componente	Función
Figure	Contenedor principal del gráfico
Axes	Área de dibujo con sus propios ejes
Axis	Líneas que representan los ejes X y Y
Ticks	Marcas numéricas en los ejes
Labels	Nombres de los ejes
Legend	Descripción de las series representadas
Title	Título general del gráfico

Crear un gráfico con las funciones pyplot

El enfoque basado en pyplot es ideal para tareas rápidas. Las funciones se ejecutan de manera secuencial, lo que hace que el flujo de trabajo sea muy directo. Por ejemplo, si se quiere representar la evolución de una variable a lo largo del tiempo, se puede hacer con unas pocas líneas de código.

Este método es muy útil para análisis exploratorios o presentaciones simples. Sin embargo, para casos más complejos se recomienda usar el modelo orientado a objetos.



Crear un gráfico con modelo de orientación a objetos

Este enfoque permite mayor modularidad y reutilización de código. Es especialmente útil cuando se quieren crear múltiples subgráficos dentro de una misma figura o cuando se necesita manipular de forma independiente varios elementos del gráfico.

```
fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_title("Gráfico 00")
ax.set_xlabel("Tiempo")
ax.set_ylabel("Valor")
plt.show()
```

Figuras y subgráficos

Una de las funcionalidades más potentes de Matplotlib es la posibilidad de generar varios gráficos en una misma figura. Esto se logra usando la función `subplots`, la cual permite dividir el área total en múltiples secciones.

Por ejemplo, para mostrar dos tipos de gráficos uno al lado del otro, se puede usar:

Comparación de enfoques

Enfoque pyplot

- Más sencillo y rápido de implementar
- Ideal para análisis exploratorio
- Menos código para gráficos simples
- Estilo similar a MATLAB

Enfoque orientado a objetos

- Mayor control sobre cada elemento
- Mejor para gráficos complejos
- Facilita la reutilización de código
- Permite manipulación independiente de elementos

Ambos métodos pueden generar los mismos resultados visuales, pero el modelo orientado a objetos es más adecuado cuando se necesita personalizar cada parte del gráfico o incorporar múltiples elementos interactuando entre sí.

Ejemplo de subgráficos

La posibilidad de compartir ejes (sharex, sharey) facilita la comparación entre los gráficos, y el uso del parámetro figsize permite definir dimensiones acordes al espacio de presentación.

```
fig, axs = plt.subplots(1, 2, figsize=(10, 4))  
axs[0].plot(x, y)  
axs[1].bar(x, y)
```

Posición y tamaño de los gráficos

El tamaño de una figura no solo influye en su legibilidad, sino también en su adecuación a diferentes medios. Se puede definir desde la creación de la figura con el parámetro `figsize` en `plt.figure()` o `plt.subplots()`.

Además, si es necesario ajustar la distribución del espacio entre subgráficos, se puede utilizar `plt.tight_layout()` o `plt.subplots_adjust()` para evitar superposición de etiquetas o ejes.

Colores, marcadores y estilos



Colores

Matplotlib ofrece una amplia gama de colores predefinidos y la posibilidad de usar códigos hexadecimales para personalización exacta.



Marcadores

Diferentes formas como círculos, cuadrados, estrellas para representar puntos de datos específicos.



Estilos de línea

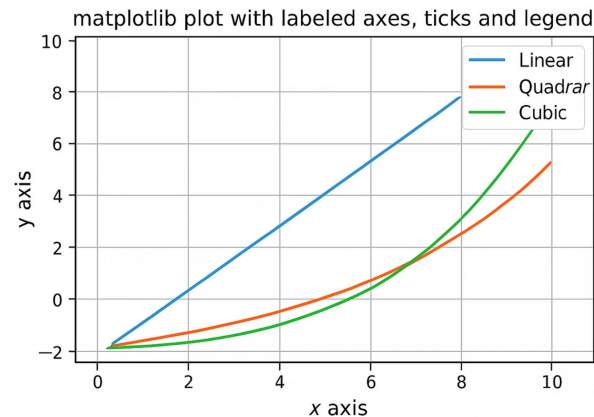
Líneas continuas, discontinuas, punteadas y combinaciones para diferenciar series de datos.

Personalizar los elementos visuales del gráfico es fundamental para comunicar claramente los datos. Estos parámetros no solo cumplen una función estética, sino también funcional, ya que permiten diferenciar múltiples series de datos o destacar tendencias.

Ticks, etiquetas y leyendas

La comprensión de un gráfico depende en gran medida de lo claras que sean sus etiquetas y leyendas. Con `xticks()` y `yticks()` es posible modificar las marcas de los ejes, mientras que con `legend()` se identifican distintas series representadas.

Agregar etiquetas comprensibles y una leyenda bien ubicada mejora significativamente la interpretación del gráfico, especialmente en contextos donde se presentan múltiples variables.



Fijando los límites de un gráfico

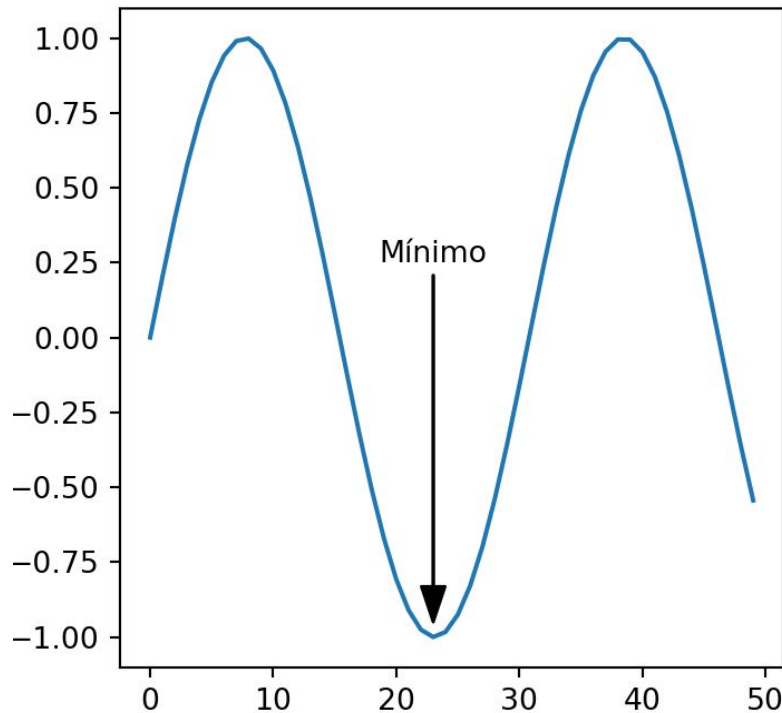
```
# Ejemplo de código para fijar límitesplt.xlim(0, 10)plt.ylim(-5, 5)
```

Es posible controlar el rango de valores visibles mediante las funciones `xlim()` y `ylim()`. Esto es útil para enfocar la atención del lector en un intervalo específico, eliminar valores atípicos o comparar gráficos entre sí con escalas comunes.

Anotaciones y dibujos en un gráfico

Matplotlib permite agregar comentarios o señales visuales dentro del gráfico usando `annotate()` o `text()`. Esto puede incluir flechas, recuadros o simples etiquetas que expliquen puntos importantes o anomalías en los datos.

Este tipo de elementos aporta contexto y facilita la comprensión por parte del usuario final.



Guardando los gráficos en un archivo

Formatos disponibles

- PNG (imágenes rasterizadas)
- PDF (documentos vectoriales)
- SVG (gráficos vectoriales)
- JPG (fotografías)
- EPS (PostScript encapsulado)

Parámetros importantes

- dpi: resolución en puntos por pulgada
- bbox_inches: control de márgenes
- transparent: fondo transparente
- facecolor: color de fondo

Una vez creado un gráfico, puede guardarse fácilmente con `savefig()`. Guardar los gráficos permite integrarlos en informes, presentaciones o documentos académicos sin pérdida de calidad.

Tipos de gráficos

Matplotlib ofrece una amplia variedad de tipos de gráficos para representar diferentes tipos de datos y relaciones. La elección del tipo de gráfico adecuado depende del mensaje que se quiera transmitir y de la naturaleza de los datos.

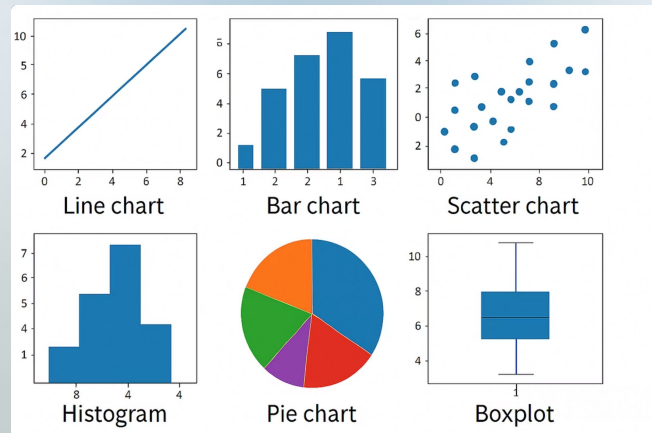
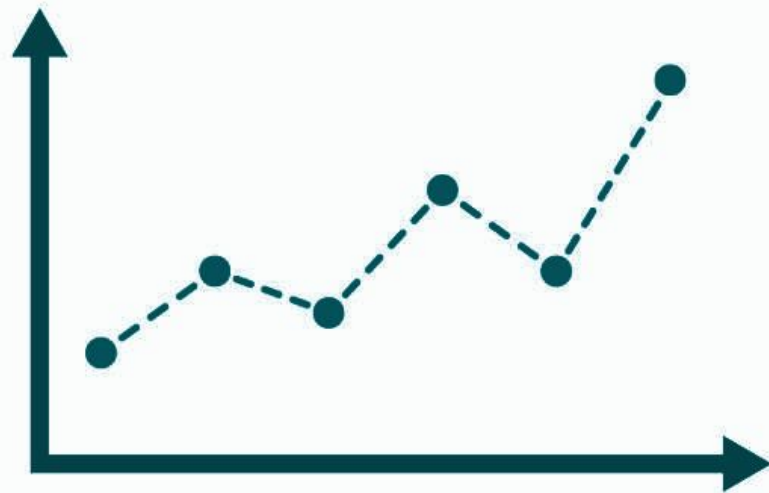


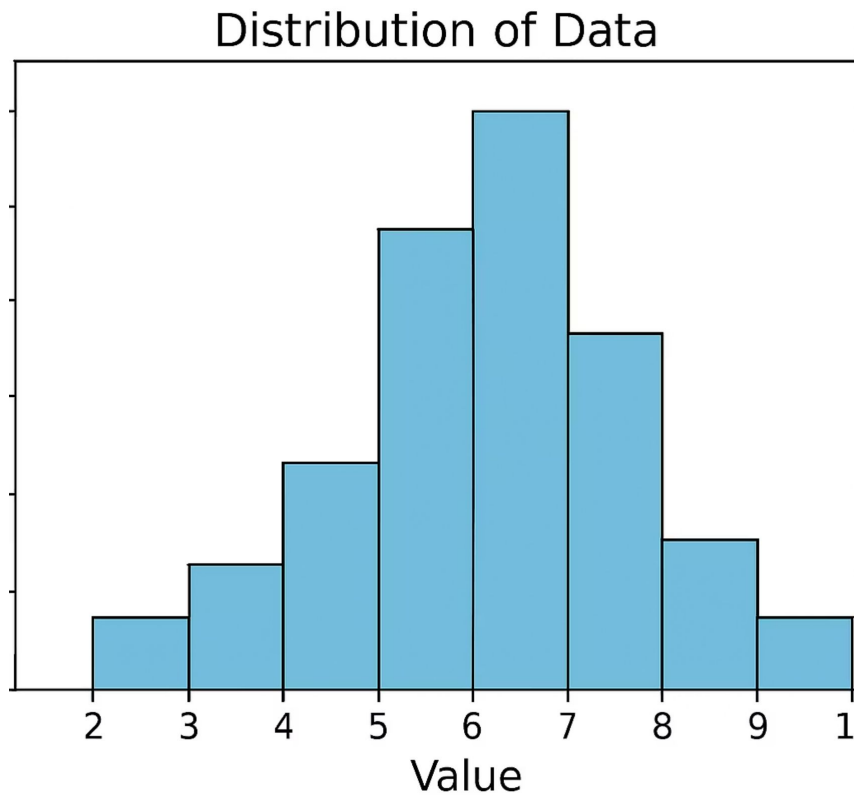
Diagrama de línea

Es el más usado para representar series temporales o secuencias ordenadas. Su forma continua permite visualizar tendencias o variaciones a lo largo del tiempo.

```
plt.plot(x, y)plt.title('Diagrama de línea')plt.xlabel('Eje X')plt.ylabel('Eje Y')plt.show()
```



Histograma



Permite ver la distribución de frecuencias de un conjunto de datos. Es útil para detectar asimetrías, acumulaciones o

```
plt.hist(data,  
bins=20)plt.title('Histograma')plt.xlabel('Valores')plt.  
t.ylabel('Frecuencia')plt.show()
```

Boxplot

Muestra la dispersión de los datos, incluyendo la mediana, los cuartiles y posibles valores atípicos. Es ideal para análisis

```
plt.boxplot(data)plt.title('Boxplot')plt.xlabel('Grupos')plt.ylabel('Valores')plt.show()
```

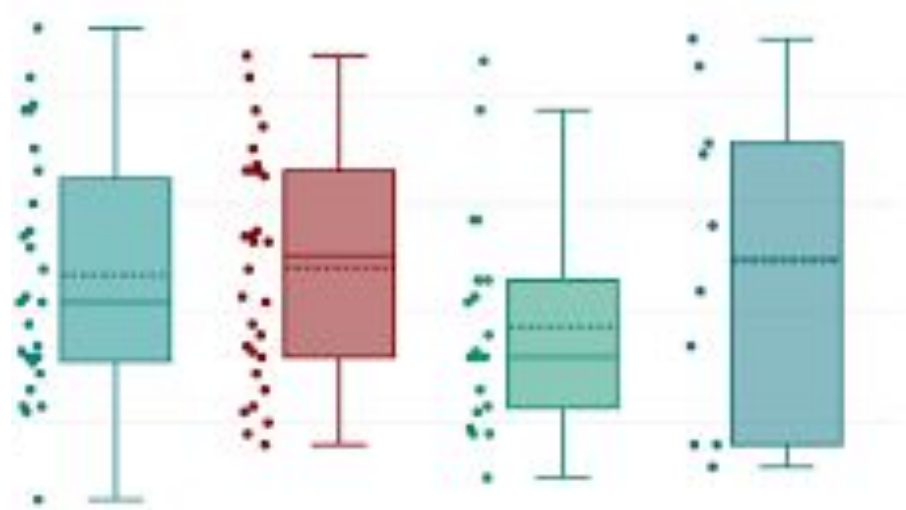
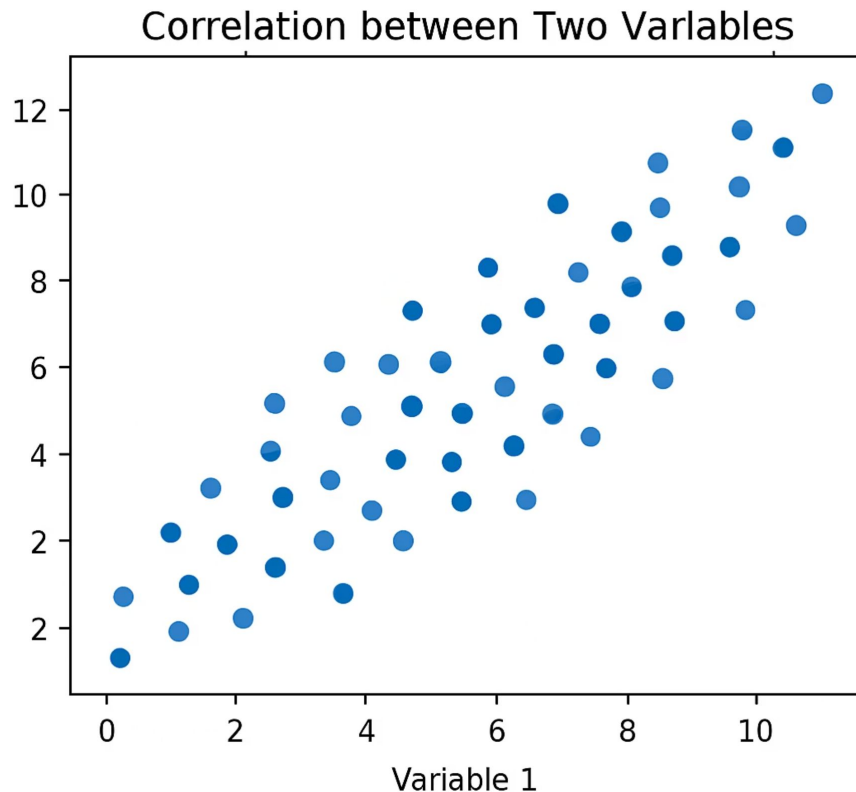


Diagrama de dispersión



Representa dos variables numéricas en un plano cartesiano, permitiendo detectar correlaciones o agrupamientos.

```
plt.scatter(x, y)plt.title('Diagrama de  
dispersión')plt.xlabel('Variable X')plt.ylabel('Variable  
Y')plt.show()
```

Diagrama de barras

Representa valores categóricos. Es eficaz para comparar cantidades entre distintos grupos.

```
plt.bar(categories, values)plt.title('Diagrama de  
barras')plt.xlabel('Categorías')plt.ylabel('Valores')plt.show()
```

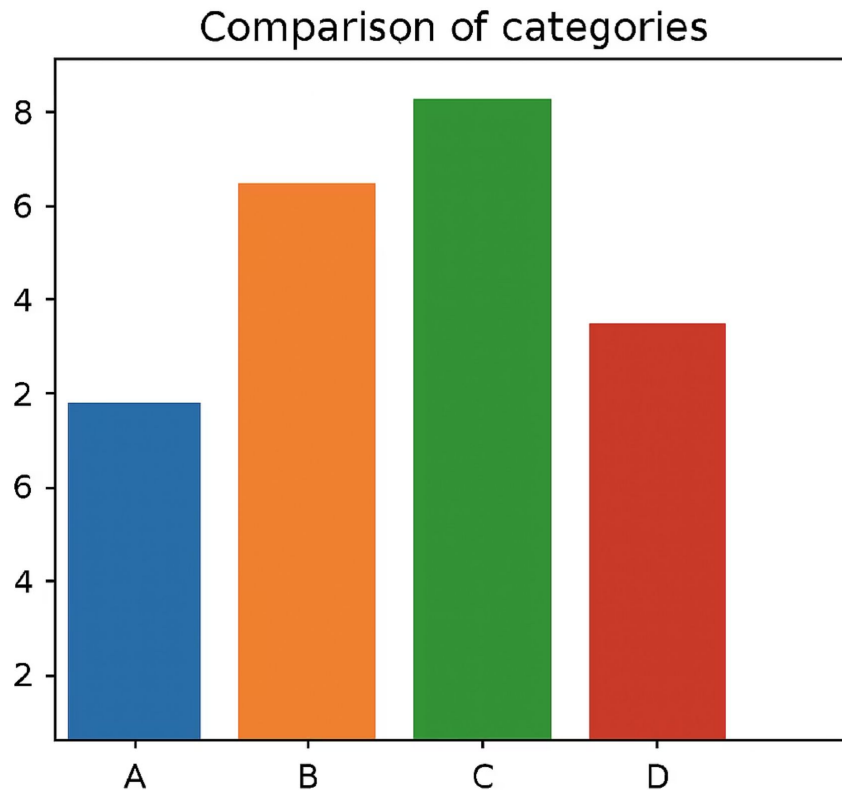
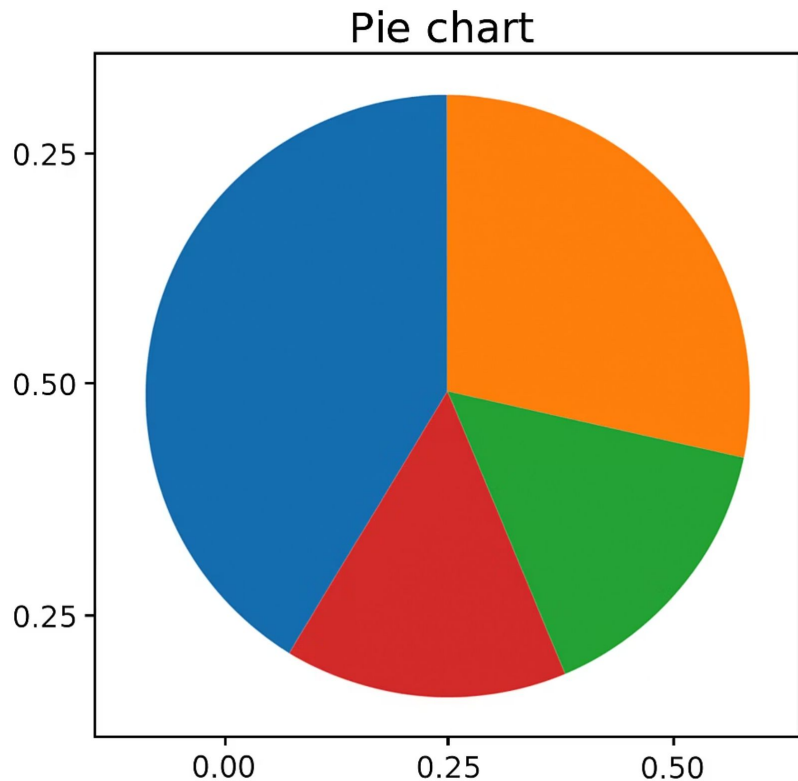


Diagrama de torta



Muestra proporciones relativas de un total. Se recomienda usarlo solo con pocos valores, ya que su legibilidad disminuye

```
plt.pie(sizes, labels=labels)plt.title('Diagrama de  
torta')plt.axis('equal')plt.show()
```

Personalización avanzada de gráficos

Estilos predefinidos

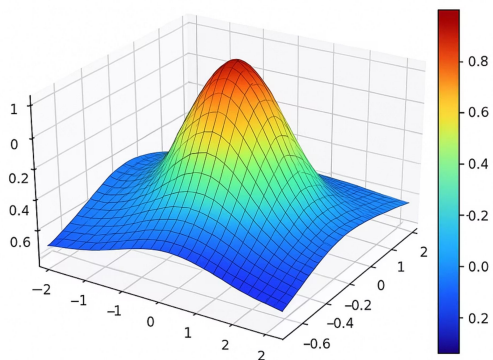
Matplotlib incluye varios estilos predefinidos como 'ggplot', 'seaborn', 'dark_background' que pueden aplicarse con `plt.style.use('nombre_estilo')`.

Personalización de fuentes

Es posible modificar el tipo, tamaño y estilo de las fuentes para mejorar la legibilidad o adaptarse a requisitos específicos de publicación.

Paletas de colores

Existen paletas predefinidas para diferentes tipos de datos (secuenciales, divergentes, cualitativos) que ayudan a representar mejor la información.



Gráficos 3D con Matplotlib

Aunque Matplotlib es principalmente una biblioteca para visualización 2D, también ofrece capacidades para crear gráficos tridimensionales a través del módulo `mplot3d`. Esto permite representar superficies, líneas y puntos en un espacio tridimensional.

Integración con otras bibliotecas



NumPy

Proporciona estructuras de datos eficientes para manipulación numérica que Matplotlib puede graficar directamente.



Pandas

Ofrece métodos integrados que utilizan Matplotlib para visualizar DataFrames y Series con una sintaxis simplificada.

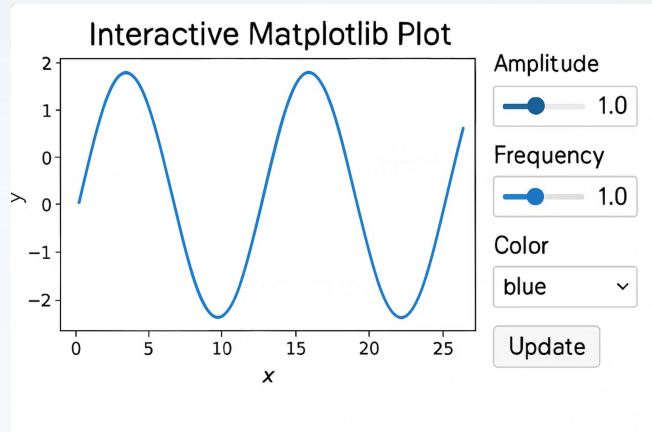


Seaborn

Construida sobre Matplotlib, proporciona una interfaz de alto nivel para crear gráficos estadísticos más atractivos.

Gráficos interactivos

Aunque Matplotlib no fue diseñado originalmente para interactividad, se puede combinar con herramientas como ipywidgets en Jupyter Notebooks para crear visualizaciones dinámicas que respondan a entradas del usuario.



Optimización de rendimiento

Uso de blitting

Técnica que actualiza solo las partes del gráfico que han cambiado, útil para animaciones fluidas.

Reducción de elementos

Limitar el número de puntos o líneas cuando se trabaja con conjuntos de datos muy grandes.

Backend adecuado

Seleccionar el backend más eficiente según el entorno de trabajo (Qt, TkAgg, etc.).



Momento:

Time-out!



5 -10 min.





Ejercicio

Visualizando una población de estudiantes

Visualizando una población de estudiantes

Contexto: 🙌

Estás colaborando con un departamento académico que necesita presentar un resumen visual de su cohorte de estudiantes. Para esto, es necesario representar datos como edad, promedio y materias cursadas usando Matplotlib.

Consigna: ✍️

Simule datos de al menos 50 estudiantes. Usando Matplotlib, crea:

- Un histograma de edades.
- Un boxplot de promedios.
- Un gráfico de líneas de evolución del promedio por edad.
- Personalización completa de etiquetas, color y leyendas.

Tiempo 🕒: 45 Minutos

Paso a paso: ⚙️

1. Cree listas de edades (18 a 30) y promedios (1 a 10).
2. Use `plt.hist()` y `plt.boxplot()` para representar ambas variables.
3. Añade etiquetas, títulos y colores personalizados.
4. Muestre los gráficos en subplots.
5. Guarde el gráfico en formato PNG.

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Implementamos gráficos básicos con Matplotlib (line, hist, boxplot).
- ✓ Aplicamos personalización visual a títulos, colores y estilos.
- ✓ Combinamos subgráficos en una misma figura usando subplot.
- ✓ Simulamos datos para visualizar situaciones reales.
- ✓ Exportamos gráficos para presentaciones.



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

